

Analyse C1 et C2 de l'API bancaire JavaScript

Complexité cyclomatique et graphes de flot de contrôle

Projet : API bancaire JavaScript

Étudiant : AMAYEN BOUSSIOM THIERRY MARTIN

Matricule : 25I2177

Dépôt GitHub : AmayenICTL3/TPICT304

Code source analysé : `server.js`

1 Fonctionnalités du logiciel

Les cinq fonctions principales étudiées à partir du code source sont :

1. créer un compte bancaire ;
2. lister les comptes ;
3. consulter un compte ;
4. effectuer un dépôt ;
5. effectuer un retrait.

Le projet implémente aussi deux fonctions supplémentaires : l'historique des transactions et le virement entre comptes.

2 Rappel sur C1 et C2

C1 : couverture des instructions. Chaque instruction doit être exécutée au moins une fois.

C2 : couverture des décisions. Chaque décision doit être exercée au moins une fois à vrai et au moins une fois à faux.

3 Complexité cyclomatique

La complexité cyclomatique est calculée avec la forme simplifiée :

$$M = D + 1$$

où D est le nombre de décisions du graphe.

Fonction	Décisions	Complexité M	Interprétation
Créer un compte	3	4	Champs obligatoires, solde négatif, dépôt initial.
Lister les comptes	0	1	Chemin séquentiel unique.
Consulter un compte	1	2	Compte trouvé ou introuvable.
Effectuer un dépôt	2	3	Compte introuvable, montant invalide ou dépôt valide.
Effectuer un retrait	3	4	Compte introuvable, montant invalide, solde insuffisant ou retrait valide.

4 Graphes de flot de contrôle

4.1 Créer un compte

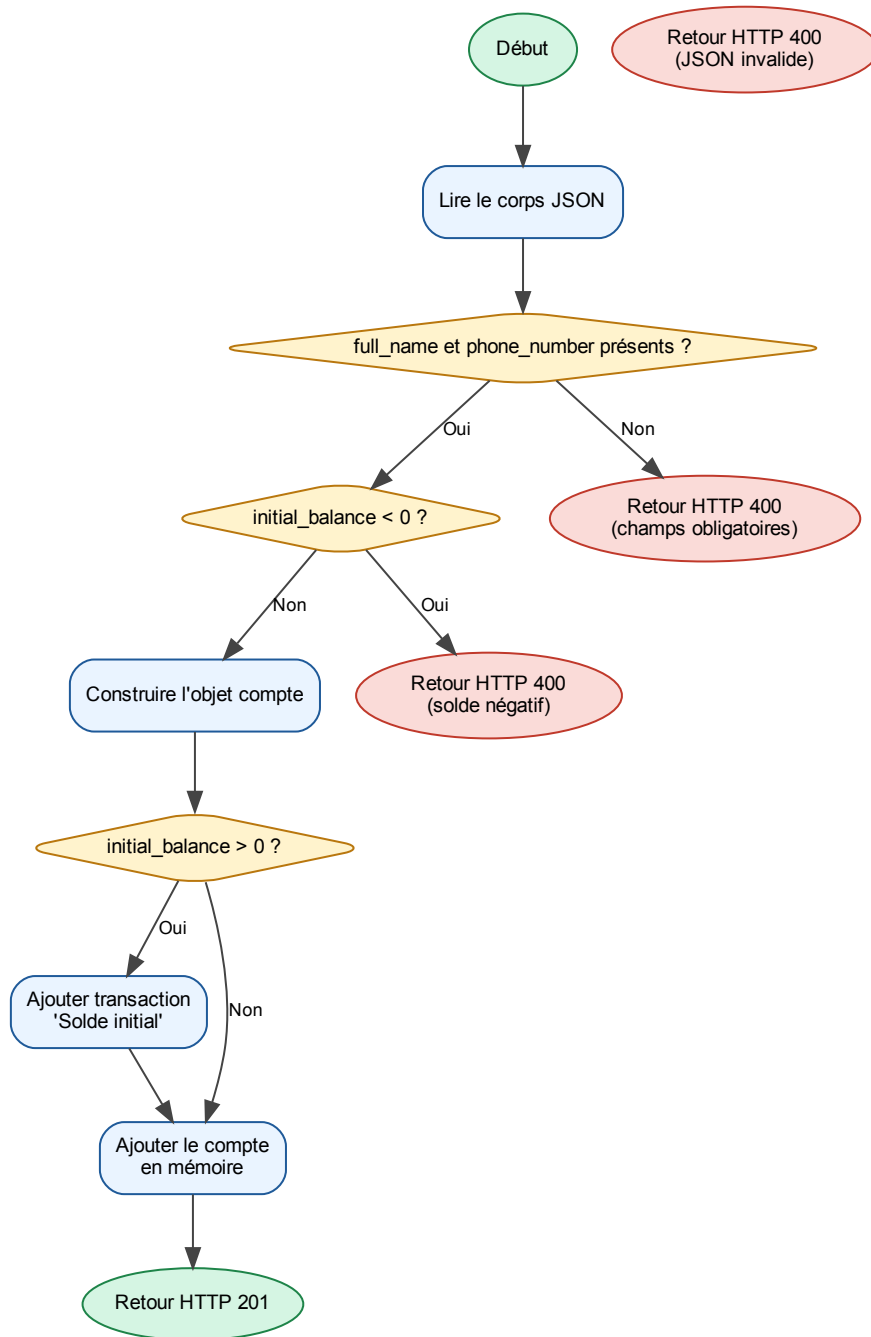


FIGURE 1 – Graphe de flot de contrôle de la création de compte

Chemins principaux

- P1 : champs obligatoires absents → erreur HTTP 400.
- P2 : solde initial négatif → erreur HTTP 400.
- P3 : création sans solde positif → compte créé sans transaction initiale.
- P4 : création avec solde positif → transaction **deposit** initiale ajoutée.

4.2 Lister les comptes

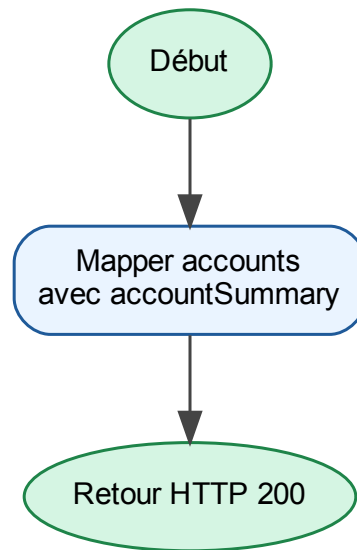


FIGURE 2 – Graphe de flot de contrôle de la liste des comptes

Chemin principal

- P1 : lecture du tableau `accounts` puis retour HTTP 200.

4.3 Consulter un compte

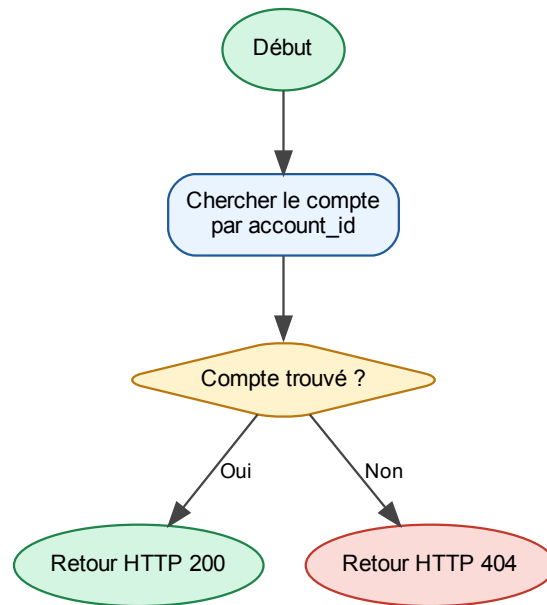


FIGURE 3 – Graphe de flot de contrôle de la consultation d'un compte

Chemins principaux

- P1 : compte trouvé → retour HTTP 200.
- P2 : compte introuvable → retour HTTP 404.

4.4 Effectuer un dépôt

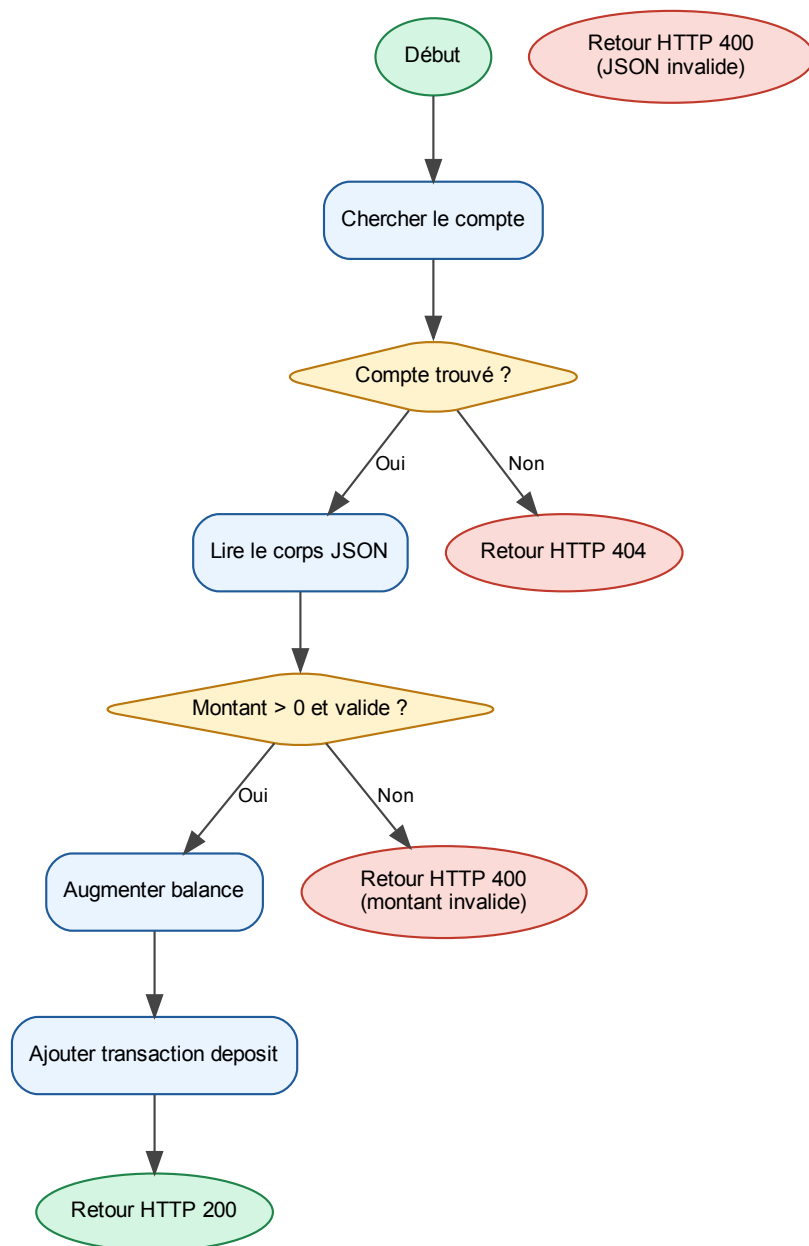


FIGURE 4 – Graphe de flot de contrôle du dépôt

Chemins principaux

- P1 : compte introuvable → HTTP 404.
- P2 : montant invalide ou nul → HTTP 400.
- P3 : dépôt valide → solde augmenté et transaction enregistrée.

4.5 Effectuer un retrait

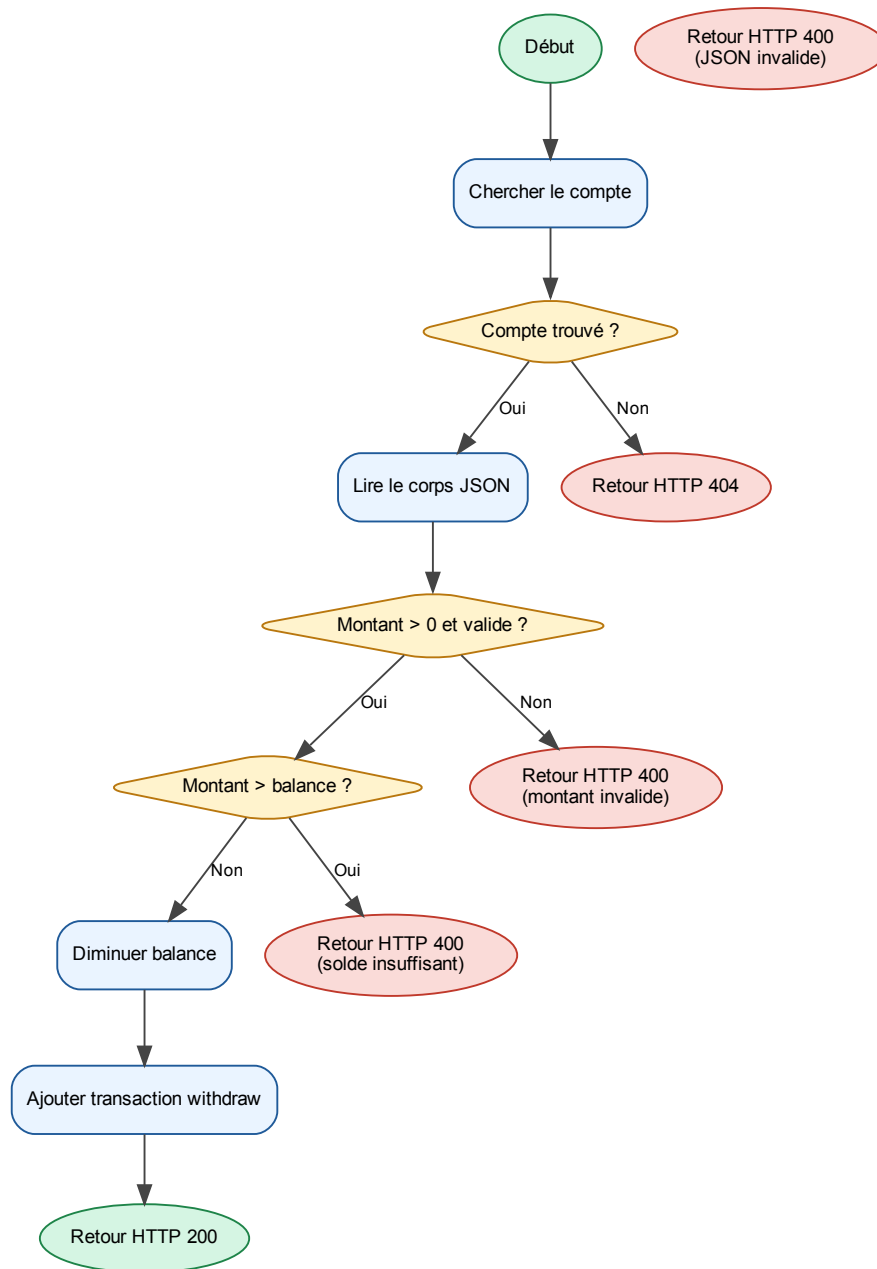


FIGURE 5 – Graphe de flot de contrôle du retrait

Chemins principaux

- P1 : compte introuvable → HTTP 404.
- P2 : montant invalide → HTTP 400.
- P3 : solde insuffisant → HTTP 400.
- P4 : retrait valide → solde diminué et transaction enregistrée.

5 Table de progression de couverture

Fonction	Tests minimaux pour C1	Tests minimaux pour C2	Résultat
Créer un compte	création sans solde + création avec solde + erreur champs + erreur solde négatif	idem	C1/C2 atteints
Lister les comptes	appel simple GET	appel simple GET	C1 atteint
Consulter un compte	compte existant + compte introuvable	idem	C1/C2 atteints
Effectuer un dépôt	dépôt valide + montant invalide + compte introuvable	idem	C1/C2 atteints
Effectuer un retrait	retrait valide + montant invalide + solde insuffisant + compte introuvable	idem	C1/C2 atteints

6 Conclusion

Cette analyse montre que les fonctions de dépôt et surtout de retrait contiennent plusieurs décisions importantes pour les tests structurels. Le retrait est plus complexe parce qu'il faut vérifier l'existence du compte, la validité du montant et la suffisance du solde.

Grâce aux graphes visuels et à la complexité cyclomatique, on peut déterminer clairement les chemins indépendants à couvrir et justifier les cas de test nécessaires pour les critères C1 et C2.